LEARN BEING FORWARD

Programming — Java Programming

Core Java syntax & OOP

COMPLETE LEARNING GUIDE

What You'll Learn

Java is a statically-typed, object-oriented language that compiles to platform-independent bytecode run by the Java Virtual Machine, and nearly every enterprise backend, Android app, and large-scale system you'll touch in a placement-ready career rests on the mechanics covered here. This sub-module builds Java from the ground up: how a source file becomes a running program, the primitive type system and operators, modern control-flow syntax including switch expressions, and the four pillars of object-oriented programming — encapsulation, inheritance, polymorphism, and abstraction — each demonstrated with real, compilable code. Getting these fundamentals genuinely solid matters because every later topic (collections, exceptions, concurrency) is really just OOP applied to a specific problem, and most 'confusing' Java bugs trace back to a shaky grasp of one of these basics rather than anything exotic.

1. Program Structure, Compilation, and the JVM

A Java program starts as one or more .java source files, each typically containing one public class whose name must exactly match the filename (Hello.java must contain public class Hello). The javac compiler translates this source into platform-independent .class bytecode, which the Java Virtual Machine (JVM) then interprets or JIT-compiles into native instructions at runtime — this is what 'write once, run anywhere' actually means: the same bytecode runs unmodified on any machine with a compatible JVM. Execution always begins at the public static void main(String[] args) method, which the JVM looks for by exact signature. Classes are organized into packages (declared with a package statement at the top of the file) mirroring the folder structure, and import statements bring other packages' classes into scope.

- The public class name and the .java filename must match character-for-character, including case.
- main's signature is fixed: public static void main(String[] args) — the JVM won't find any other signature.
- Bytecode portability is why a .class file compiled on Windows runs unmodified on Linux or macOS.

```java
package com.example.intro;

public class Hello {
    public static void main(String[] args) {
        System.out.println("Hello, Java!");
    }
}
// Compile:  javac Hello.java   -> produces Hello.class
// Run:      java com.example.intro.Hello
```

2. Variables, Primitive Types, and Operators

Java has exactly eight primitive types — byte, short, int, long, float, double, char, and boolean — each with a fixed size and a default value, which is why a declared-but-unassigned instance field of type int defaults to 0 rather than causing an error. Everything else (String, arrays, your own classes) is a reference type, stored as a pointer to heap-allocated data rather than the value itself. Arithmetic, relational, logical, and the ternary (?:) operators behave largely like other C-family languages, but integer division truncates toward zero silently — 7 / 2 evaluates to 3, not 3.5 — which is one of the first surprises most learners hit. Widening casts (int to double) happen automatically; narrowing casts (double to int) require an explicit cast and truncate the decimal portion rather than rounding.

- int / int always produces an int, truncating any remainder — cast at least one operand to double to get a fractional result.
- char arithmetic (e.g., 'a' + 1) promotes to int automatically, so the result is 98, not 'b'.
- Primitive variables hold their value directly; reference variables hold a pointer to an object on the heap.

```java
int total = 7;
int count = 2;
System.out.println(total / count);           // 3 — integer division truncates
System.out.println((double) total / count);  // 3.5 — cast forces floating-point division

char letter = 'a';
int shifted = letter + 1;                     // 98, promoted to int
System.out.println((char) shifted);           // 'b' — must cast back explicitly
```

3. Control Flow: Conditionals, Loops, and Modern switch

if/else if/else chains and the classic C-style switch (which falls through between cases without an explicit break) work as in most languages, but Java 14 standardized a far safer switch expression syntax using arrows (case X -> result) that doesn't fall through, can directly return a value, and supports yield for multi-statement branches. Loop constructs cover for, while, do-while (which guarantees at least one execution of the body), and the enhanced for-each loop for iterating arrays and collections without manual indexing. break and continue can be combined with labels to control outer loops from inside nested ones, which is occasionally the cleanest way to exit a doubly-nested search loop.

- Classic switch falls through to the next case without an explicit break — a frequent source of bugs when a case is added later.
- Switch expressions (case X -> ...) are safer by default and can be assigned directly to a variable.
- do-while always executes its body at least once, even if the condition is false from the start.

```java
int day = 3;

// Modern switch expression — no fallthrough, returns a value directly
String dayName = switch (day) {
    case 1, 7 -> "Weekend";
    case 2, 3, 4, 5, 6 -> "Weekday";
    default -> "Invalid day";
};
System.out.println(dayName); // "Weekday"
```

4. Classes, Objects, and Constructors

A class is a blueprint describing the fields (state) and methods (behavior) an object will have; an object is a concrete instance created with the new keyword, which allocates memory on the heap and returns a reference to it. Constructors initialize a new object's state and share the class's name with no return type; a class can have multiple constructors with different parameter lists (constructor overloading), and one constructor can delegate to another in the same class using this(...) as its first statement. If no constructor is written at all, Java silently provides a no-argument default constructor — but that disappears the moment you write any constructor yourself, a detail that trips up many learners who then can't call new MyClass() anymore.

```java
public class Person {
    private final String name;
    private int age;

    public Person(String name) {
        this(name, 0); // delegate to the other constructor
    }

    public Person(String name, int age) {
        this.name = name;
        this.age = age;
    }

    public String describe() {
        return name + " is " + age + " years old";
    }
}

Person p = new Person("Asha", 22);
System.out.println(p.describe());
```

5. Encapsulation and Access Modifiers

Encapsulation means hiding an object's internal state behind a controlled interface — fields are declared private and exposed only through public getter and setter methods, which lets a class validate or transform data on the way in and change its internal representation later without breaking any code that uses it. Java has four access levels: private (visible only within the same class), default/package-private (no modifier — visible within the same package), protected (visible within the package plus subclasses elsewhere), and public (visible everywhere). A setter that validates its input — rejecting a negative age, for instance — is the entire practical point of encapsulation: without it, any code anywhere could silently put an object into an invalid state.

- Default (no modifier) access is package-private, not 'private by default' as some learners assume.
- A setter is a natural place to enforce invariants (e.g., throwing an exception on a negative age) that a public field could never enforce.

```java
public class BankAccount {
    private double balance;

    public void deposit(double amount) {
        if (amount <= 0) {
            throw new IllegalArgumentException("Deposit must be positive");
        }
        balance += amount;
    }

    public double getBalance() {
        return balance;
    }
}
```

6. Inheritance: extends, super, and the Object Class

A class inherits fields and methods from a single parent using extends — Java deliberately disallows multiple class inheritance to avoid the ambiguity it creates, a limitation interfaces exist partly to work around. The super keyword calls the parent's constructor (super(...), which must be the first statement if used) or invokes an overridden parent method explicitly (super.methodName()). Every class implicitly extends Object if no other parent is specified, which is why every object automatically has toString(), equals(), and hashCode() methods available — ones that are usually worth overriding, since the inherited defaults compare by memory reference and print an unhelpful class-name-plus-hashcode string.

```java
public class Animal {
    protected String name;
    public Animal(String name) { this.name = name; }
    public String speak() { return name + " makes a sound"; }
}

public class Dog extends Animal {
    public Dog(String name) { super(name); }

    @Override
    public String speak() {
        return super.speak() + " — specifically, a bark!";
    }
}

Animal a = new Dog("Rex");
System.out.println(a.speak()); // "Rex makes a sound — specifically, a bark!"
```

7. Polymorphism: Overriding vs Overloading

Overloading is compile-time polymorphism: multiple methods share a name but differ in parameter list (type, count, or order), and the compiler picks the matching version based on the arguments at the call site. Overriding is runtime polymorphism: a subclass redefines a method with the exact same signature it inherited, and which version actually runs is decided at runtime based on the object's real (dynamic) type, not the type of the reference variable pointing to it — this is why an Animal-typed reference holding a Dog object calls Dog's overridden method, not Animal's. The @Override annotation doesn't change behavior but makes the compiler verify you actually matched a real parent method, catching signature typos that would otherwise silently create an unrelated overload instead of an override.

- Overloading is resolved at compile time from the argument types; overriding is resolved at runtime from the object's actual class.
- Always add @Override on an intended override — the compiler will error if the signature doesn't actually match a parent method.

8. Abstraction: Abstract Classes vs Interfaces

Abstraction means defining what an object can do without necessarily specifying every detail of how. An abstract class (marked abstract, cannot be instantiated directly) can mix fully-implemented methods with abstract ones that subclasses must fill in, and can hold shared state — but a class can extend only one abstract class. An interface declares a contract of methods any implementing class must provide, and since Java 8 can also include default methods (with a body, inherited unless overridden) and static methods — crucially, a class can implement any number of interfaces, which is how Java approximates multiple inheritance of behavior. The practical rule: reach for an abstract class when subclasses share common state and implementation; reach for an interface when you're defining a capability unrelated classes can all plug into.

```java
interface Drawable {
    void draw(); // abstract by default
    default String describe() { return "A drawable shape"; } // Java 8+
}

abstract class Shape implements Drawable {
    protected String color;
    Shape(String color) { this.color = color; }
    abstract double area(); // subclasses must implement
}

class Circle extends Shape {
    double radius;
    Circle(String color, double radius) { super(color); this.radius = radius; }
    @Override double area() { return Math.PI * radius * radius; }
    @Override public void draw() { System.out.println(color + " circle, area=" +
area()); }
}
```

Common Pitfalls

- Forgetting that the public class name must exactly match the .java filename, including case, which fails compilation.
- Expecting int / int to produce a decimal result — it truncates, so at least one operand must be cast to double first.
- Confusing overloading (same name, different parameters, resolved at compile time) with overriding (same signature, resolved at runtime by actual object type).
- Writing a custom constructor and then being surprised new MyClass() no longer compiles — the implicit no-arg constructor disappears once any constructor is defined.
- Forgetting super() must be the first statement in a subclass constructor, or omitting it when the parent has no no-arg constructor.
- Making fields public instead of private with validating getters/setters, defeating encapsulation and allowing invalid object state.
- Attempting to instantiate an abstract class or interface directly with new instead of a concrete implementing subclass.
- Comparing objects with == instead of .equals(), which checks reference identity rather than logical equality unless equals() is overridden.

Key Takeaways

- javac compiles to portable bytecode; the JVM is what makes 'write once, run anywhere' true.
- Integer division truncates toward zero — cast to double explicitly whenever a fractional result is needed.

- Switch expressions (case X -> ...) avoid the fallthrough bugs of classic switch statements and can return a value directly.
- Overloading is resolved at compile time from parameter types; overriding is resolved at runtime from the object's actual class.
- A class extends at most one class but can implement any number of interfaces — use abstract classes for shared state, interfaces for shared capability contracts.
- Override equals(), hashCode(), and toString() deliberately — the inherited Object defaults compare by reference and rarely print anything useful.

Learn More

Oracle — The Java Tutorials: Classes and Objects
https://docs.oracle.com/javase/tutorial/java/javaOO/index.html

Oracle — Language Basics
https://docs.oracle.com/javase/tutorial/java/nutsandbolts/index.html

Oracle JDK 17 — Switch Expressions
https://docs.oracle.com/en/java/javase/17/language/switch-expressions.html